

Freight Rate Prediction

Spotter Machine Learning Engineer Assessment: validation approach, model and December chart

1. Summary

The task is to predict **posted_rate** for 12,000 loads dated Nov–Dec 2025, using 48,000 labelled loads from Jan–Oct 2025. Because the target period lies entirely after the training data, all model decisions were made with a **forward-in-time (rolling-origin) cross-validation** rather than a random split.

Model (time-series CV, mean of 5 forward folds)	MAPE	MAE	RMSE	Bias
Baseline: median rate/mile by equipment x distance band	3.74%	\$81.4	\$110.9	-0.6%
Gradient boosting, all features (no trend)	2.72%	\$60.5	\$82.9	-1.9%
Additive GBM without trend	3.31%	\$73.5	\$95.4	-3.0%
Ridge regression on log(rate)	2.04%	\$46.0	\$65.4	+0.0%
Hybrid: GBM lane + linear market & trend	1.77%	\$39.0	\$55.7	-0.2%
Final, trained without removing outliers	1.76%	\$39.2	\$56.2	+0.2%
Final ensemble (hybrid + additive GBM)	1.71%	\$37.8	\$54.1	+0.0%

The final model reaches **1.71% MAPE / \$37.8 MAE with no systematic bias** on held-out future months. That is less than half the error of a lane rate-sheet baseline (3.74%) and well below an unconstrained gradient-boosting model (2.72%, which is biased –1.9% on future months). Metrics are computed on rows with valid labels. Section 3 explains why corrupted labels are excluded from scoring.

2. Data exploration: key findings

- **Price structure.** Rate $\approx$ distance $\times$ rate-per-mile. Rate-per-mile falls with distance, from about \$2.7/mi on short hauls to about \$1.9/mi beyond 2,000 mi. Equipment premiums over Dry Van are Reefer +12% and Flatbed +7%. Weight adds about +3% per 10,000 lb. The structure is multiplicative, so the model is fit on log(rate).
- **market_index is a date-level signal.** Its standard deviation within a date is 0.025, against 0.17 across dates. It has a strong 7-day cycle (peak Thursday, trough Sunday) on top of multi-month regimes, and daily rates follow it.
- **Trend and quarter-end effect.** After controlling for the market, rates drift up by about 7% per year. They also ramp up over the last ~30 days of every quarter (Mar, Jun, Sep), which is the familiar quarter-end tightening in trucking.
- **quote_signal** is mostly a regime indicator: its daily mean steps up in the last month of Q1–Q3 but not in Nov–Dec. Per load it carries little information.
- **Distribution shift.** The validation period sits in a low-market regime (index $\approx$ 0.92). It also contains 8 cities that never appear in training (1,447 loads, 12%).

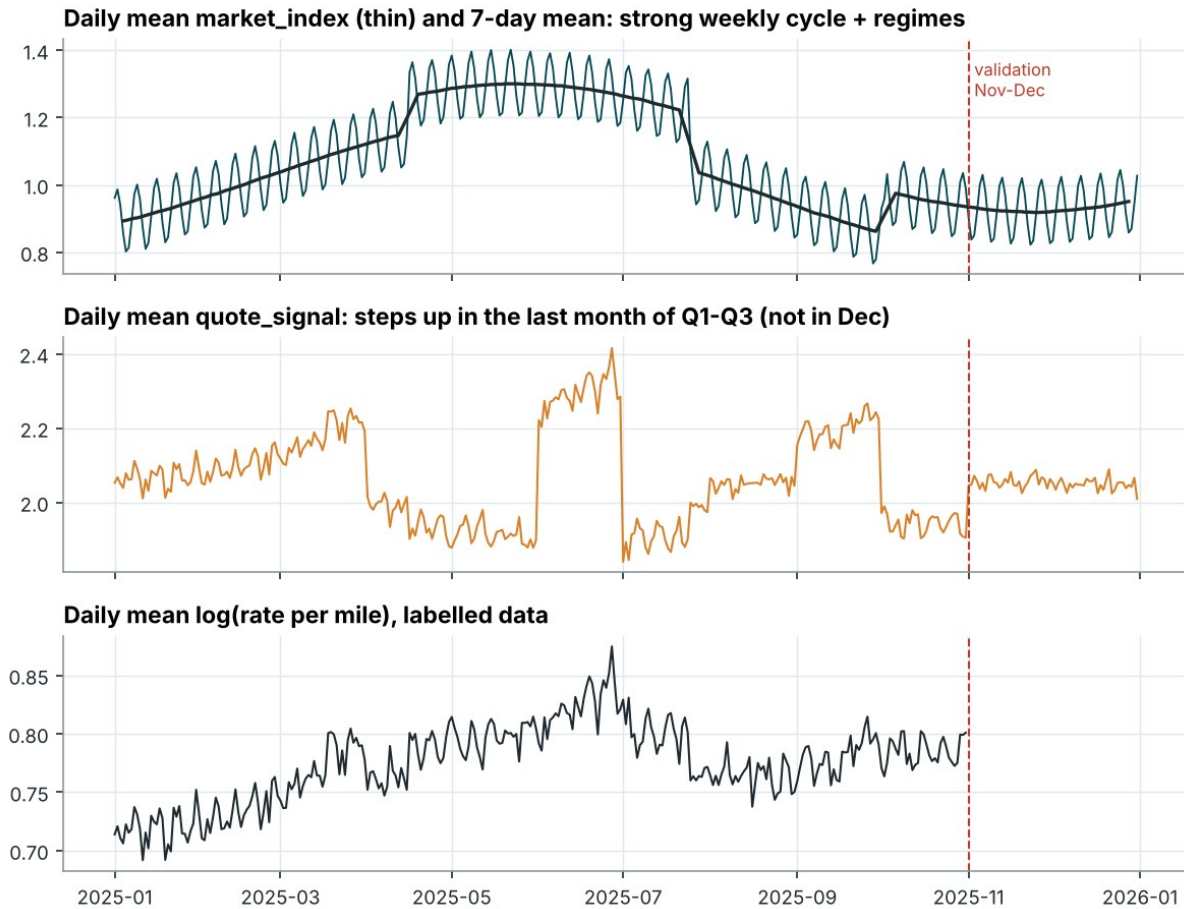


Figure 1. Daily market_index, quote_signal and average rate per mile over time. Validation starts at the dashed line.

3. Data-quality issues and how they were addressed

Issue	Rows (train / valid)	Handling
Corrupted posted_rate labels: 2-6× too high or 0.15-0.5× too low	677 (1.41%) / n.a.	Detected out-of-fold: a 5-fold model predicts every row, and labels more than 1.5× from the prediction are flagged. They are removed from training and from evaluation, and an absolute-error loss handles any that remain.
Negative weight (sign error)	292 / 145	Absolute value. After the fix, the residuals of these rows match the clean rows.
Missing weight	300 / 165	Kept as NaN (the GBM routes missing values natively), plus a missing-flag feature.
Weight piled up at 47,500 lb	1,191 / 297	Kept. This is the legal maximum, and these rows price like other heavy loads.
Missing market_index	374 / 249	Imputed with the median market_index of the other loads on the same date (date-level signal).
Cities unseen in training	n.a. / 1,447	Lanes are described by coordinates, distance and direction, not city IDs. Checked with an unseen-city CV.
Duplicate IDs or duplicate rows	0 / 0	None found.

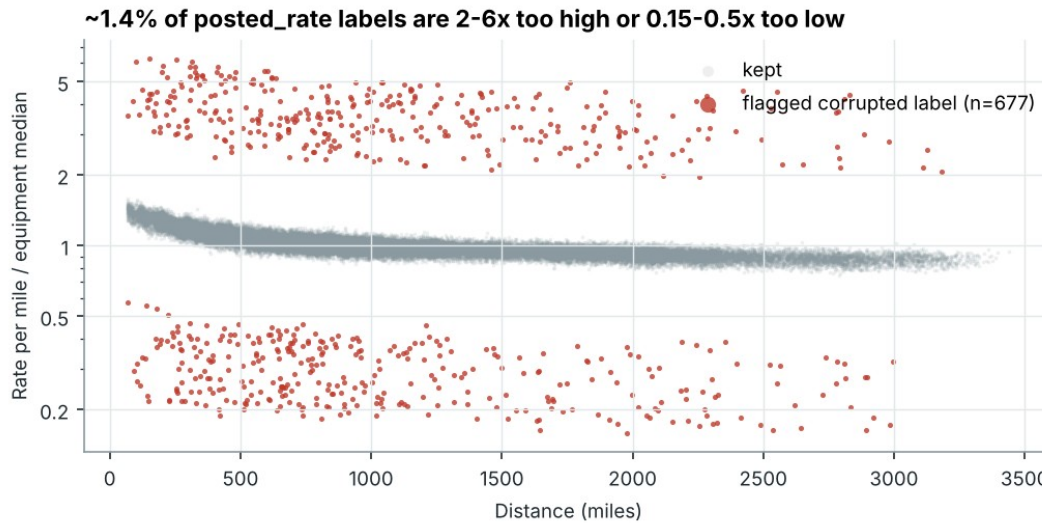


Figure 2. Rate per mile relative to the equipment median. Flagged labels sit far outside the normal pricing band.

The corrupted labels are not a plausible part of the price distribution: they fall in two clusters separated from the normal band by more than 2 $\times$. The threshold is not sensitive: every cut-off from 1.3 $\times$ to 2 $\times$ flags exactly the same 677 rows. Training with or without them changes time-CV MAPE from 1.76% to 1.71%. Validation metrics are reported on clean labels; raw metrics (including corrupted labels) are in reports/validation_results.csv.

4. Train/test split and validation approach

4.1 Why a time-based split

validation.csv (Nov–Dec) is strictly in the future relative to train_test.csv (Jan–Oct). A random K-fold would put loads from the same days and the same market regime in both train and test. It would then reward memorising daily conditions and hide the model's inability to extrapolate. The primary scheme is therefore an **expanding-window, rolling-origin CV**:

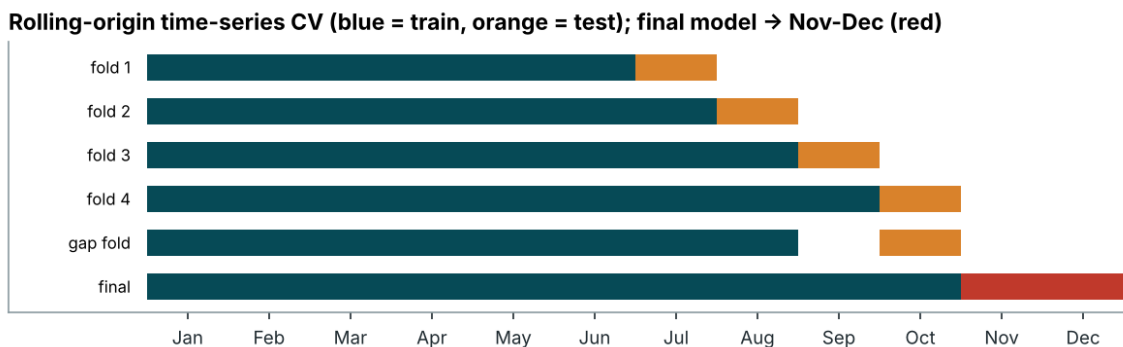


Figure 3. Rolling-origin folds. The gap fold leaves one month out between train and test, like predicting December from data that ends in October.

- Folds 1–4: train on all months before the test month, then test on Jul, Aug, Sep, Oct (about 4,700 loads each).

- Gap fold: train Jan–Aug, test Oct. December is two months beyond the training data, so this fold checks that accuracy holds with a gap.
- Daily market aggregates are computed without labels, so no target information leaks across folds. The label-outlier filter is applied to training folds only.

4.2 Secondary checks

Scheme	Purpose	Final model MAPE	Baseline MAPE
Time-series CV (primary)	Mimics the real forward prediction	1.71%	3.74%
Random 5-fold	Shows how optimistic a leaky split is	1.52%	4.27%
Unseen-city CV (8 city groups)	Generalisation to new cities	1.68%	4.31%

Random K-fold looks better for every tree model because it leaks same-day market conditions. The unconstrained GBM gets 1.34% on random folds but 2.72% on forward folds. Model selection on a random split would have picked the wrong model. Holding out entire cities barely changes the error, which confirms that the coordinate-based lane features generalise to the 8 new validation cities.

4.3 Results by fold (final model)

Fold	Test month	MAE	RMSE	MAPE	Bias
Fold 1	Jul	\$36.4	\$51.5	1.68%	+0.38%
Fold 2	Aug	\$41.6	\$62.0	1.77%	+0.89%
Fold 3	Sep	\$37.3	\$53.8	1.65%	+0.31%
Fold 4	Oct	\$36.5	\$51.3	1.70%	-0.74%
Gap fold	Oct (gap)	\$37.0	\$52.1	1.74%	-0.78%

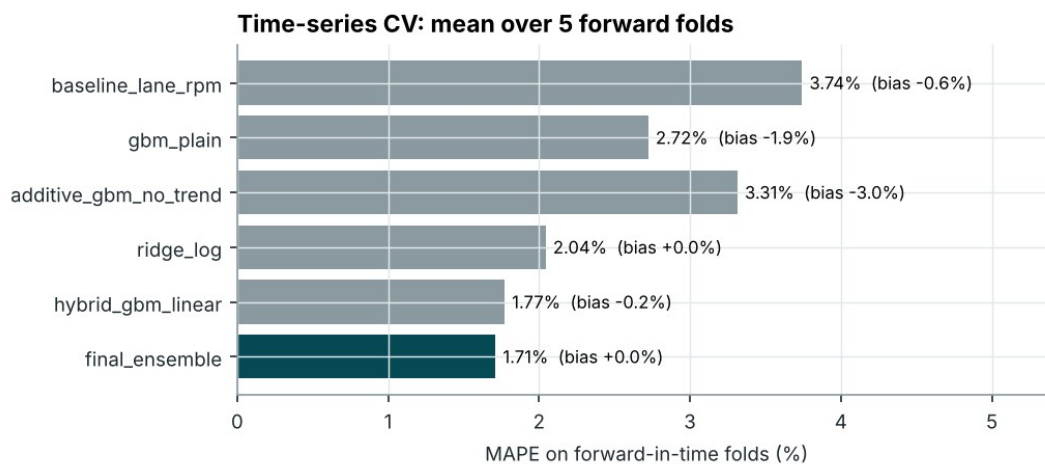


Figure 4. Model comparison on forward folds.

5. Model

The final model is the geometric mean of two **additive models in log space** that share one linear time trend:

- **Hybrid model:** gradient-boosted trees on lane/load features (distance, equipment, weight, pickup/delivery coordinates, direction, circuitry), plus a linear market component (daily market index, daily quote signal, the load's quote signal relative to the day, a quarter-end ramp and a time trend). The two parts are fit by backfitting.
- **Additive GBM:** gradient-boosted trees on a de-trended target, with interaction constraints so that no tree mixes lane features with market/calendar features.

Two problems found during validation shaped this design:

- **Trees cannot extrapolate a trend.** Every forward fold of the plain GBM was biased by about -2%. The explicit linear trend (+7%/yr) removes the bias (+0.0% on average).
- **Spurious interactions.** With only ~300 distinct days, the unconstrained GBM learnt "equipment premium × market level" interactions that did not hold in August (4.3% MAPE on that fold). Keeping lane and market effects additive brought the fold down to 1.8%.

The market coefficients of the fitted hybrid model are: daily market_index **0.141** (log-rate per index point), quarter-end ramp **0.0014** per day in the last 30 days of a quarter, and trend **+7.0%** per year. Gradient boosting (scikit-learn HistGradientBoosting) handles the non-linear distance curve and the geography with no feature scaling, and it handles missing weight natively. Absolute-error loss makes it robust to any label noise that remains. The whole pipeline trains in about 40 seconds.

6. Fixed December prediction chart

The chart template fixes the lane (Lexington → Fort Wayne, 360 mi, Dry Van, 32,000 lb) and varies only the date. Model inputs that the template lacks were filled the way they would be in production:

- Coordinates come from the city lookup built from the data.
- market_index and quote_signal are the observed daily means over all validation loads on that date. These are market-level, label-free values known on the day.

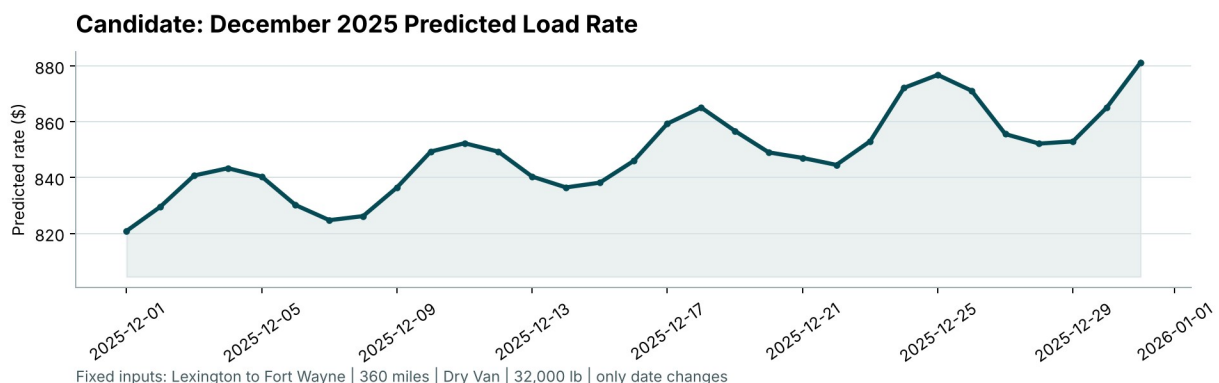


Figure 5. candidate_december.png, produced by the provided score.py.

Predictions range from **\$821** to **\$881**. The chart combines three effects. The **weekly market cycle** gives local peaks on Thursdays (4, 11, 18, 25 Dec) and troughs at weekends. The **quarter-end ramp** adds to the **upward trend**, so the weekly peaks rise by about 4% from early to late December (4 Dec vs 25 Dec), and the first-to-last-day rise is about 7%. For reference, Dry Van loads on this lane in the training data were priced at \$757–\$935, so the predictions are consistent with history once the trend is accounted for.

7. Limitations and next steps

- Holidays: the training data has no December, so no Christmas/New Year effect can be learnt beyond what `market_index` carries. Holiday indicators should be added once labelled holiday data exists.
- The trend is estimated from 10 months. It should be re-estimated regularly, or replaced by a short-window level correction, so it does not extrapolate too far.
- The corrupted labels follow a pattern that suggests systematic entry errors. They should be fixed upstream, not just filtered.
- Prediction intervals (for example quantile GBM) would let pricing teams see how uncertain each quote is.

8. Reproduction

`pip install -r requirements.txt → python scripts/train_predict.py → python score.py --predictions validation_predictions.csv --december-predictions data/december_chart_inputs.csv`. The full model comparison runs with `python scripts/validate.py`. See `README.md`.